AMPHI: Adaptive Mission-Aware Microservices Provisioning in Heterogeneous IoT Settings

Yuqiao Li*, Fangqi Liu*, Cheng-Hsin Hsu**, and Nalini Venkatasubramanian**
*Department of Computer Science, University of California Irvine, CA, {yuqiaol5, fangqil2, nalini}@uci.edu
**Department of Computer Science, National Tsing Hua University, Taiwan, {chsu}@cs.nthu.edu.tw

***Abstract*—In mission-critical IoT environments, where data from diverse data sources (stationary and mobile) must be analyzed rapidly, ensuring reliable service provisioning for the execution of analytics is critical. Under dynamic conditions and changing infrastructure capabilities, adaptive methods are required to handle surges in data volumes and execute complex services for IoT applications. In this paper, we propose AMPHI - an adaptive microservice provisioning framework that handles various mission-aware workflows in combined stationary-mobile IoT environments to enable flexible deployment of containerized microservices. AMPHI exploits the variability in cost and performance among operators that implement similar functionalities but with distinct cost/quality/time tradeoffs, and employs apriori and on-the-fly techniques for intelligent selection, placement, and sharing of operators across a hybrid set of devices (stationary sensors, drones, and rovers) to maximize Quality of Service (QoS) and resource efficiency under dynamic situations. We formulate the microservice provisioning and instantiation as an (NP-hard) optimization problem and design efficient heuristic approaches to deploy microservices by utilizing application and system context. AMPHI is evaluated in the context of smart firefighting with high-rise fires utilizing building sensors and autonomous aerial mobile units. Through real-world testbeds and extensive simulations, we show how AMPHI allows flexible and cost-effective execution of dynamically changing IoT workflows.**

***Index Terms*—IoT, microservice, edge computing, heuristic**

I. Introduction

Ubiquitous and rapidly deployable IoT systems are enabling a new level of monitoring and awareness in smart community applications [1] ranging from personalized healthcare [2] to environmental monitoring, and smart agriculture [3]. The ability to capture data anywhere, anytime, using mobile devices such as UAVs and UGVs has helped enhance situational awareness in many complex settings. Considering the case of drone-based inspection of remote infrastructure, e.g., power lines [4]–[6], or on-the-fly monitoring of emergent events in extreme and hard-to-observe scenarios, e.g., high-rise fires, flooding, earthquakes [7]–[9], near real-time data collection and processing at the network edge are required. Given geography and IoT deployment, existing infrastructure may be repurposed, and data captured may be reused for multiple time-varying applications and services. This paper addresses the efficient deployment, operation, and reconfiguration of software services on hybrid IoT devices, particularly focusing on handling time-sensitive data processing in response to changing conditions and requirements.

A popular approach to enable customizable service-oriented applications is through the use of *microservices* [10]–[12], in which a complex application is broken down into smaller, independent and loosely coupled microservices. Each microservice executes a specific functionality and operates as a separate unit within the overall application. The decoupled nature of microservices makes them easier to update, modify, or add, which in turn helps make service adjustable to adapt to various requirements. Containerization technology [13] facilitates the efficient deployment and management of microservices by encapsulating them with their dependencies into portable and isolated containers. We argue that IoT applications executing in dynamic distributed settings benefit from the flexibility offered by containerized microservices and utilize this paradigm to develop a platform for adaptive provisioning of IoT analytics for mission-critical applications.

This paper presents AMPHI, a framework for adaptively instantiating mission-aware workflows in heterogeneous IoT settings with versatile mobile and stationary devices. AMPHI receives input from IoT applications in the form of monitoring and analytics workflows, along with details about the devices available in the IoT ecosystem, including their current states of resources and capabilities. Leveraging a library of operators, AMPHI actualizes desired services by instantiating microservices and orchestrating data flows to efficiently execute multiple data analytic workflows. It generates implementation plans for microservices that align with high-level optimization objectives, such as maximizing data analysis accuracy and minimizing data processing delay. Designed for evolution, AMPHI can adapt seamlessly as the variety and scale of device clusters and operator libraries expand, thus strengthening its functionality and ensuring its capability to evolve alongside technological advancements.

The main contributions of this work include: i) Designing of AMPHI, an adaptive IoT microservice provisioning framework; ii) Formulating and solving an NP-hard mission-aware microservice provisioning problem using integer linear programming (§III and §IV); iii) Implementing AMPHI and conducting measurement studies using various stationary and mobile IoT devices (§V-A); iv) Evaluating AMPHI through both real-world testbed and large-scale simulations, with high-rise fire monitoring scenarios as a driving use case (§V-B).

II. System Overview

This section provides an overview of our hybrid edge computing environment that considers the execution of data analytics tasks on mobile and stationary nodes.

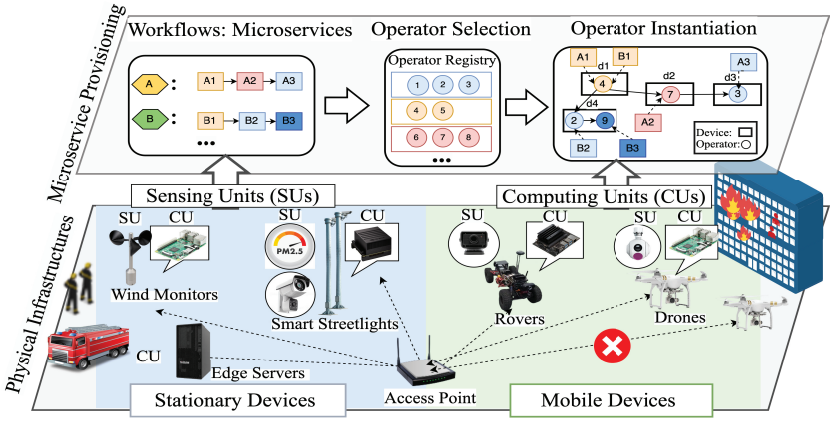


Fig. 1. Hybrid IoT infrastructures and microservice provisioning.

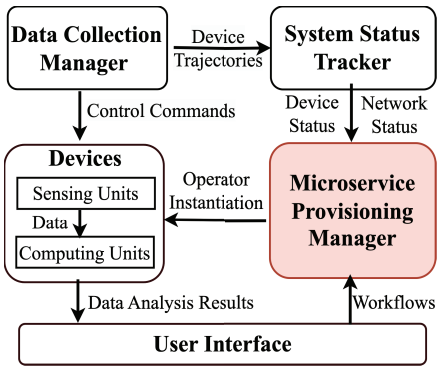


Fig. 2. Structure of AMPHI.

A. Mobile-Stationary Hybrid Edge Computing Environments

We consider heterogeneous stationary and mobile devices equipped with sensing and computing capabilities as shown in Fig. 1. These devices are connected through heterogeneous networks (e.g., WiFi, LoRa, and LTE) with varying transmission delays. Sensors embedded in devices may collect data, which could be analyzed locally or transmitted to more powerful proxy devices for analysis. Traditionally, analytics are deployed as monolithic services, which often lead to poor resource utilization and Quality of Service (QoS).

In AMPHI, we propose decomposing each data analytics service into a workflow of microservices deployed to multiple nodes, where the analytics service's results are delivered to a data sink. Such service decomposition goes beyond merely splitting a service into pieces: microservices can work independently of the original workflow; they can also composed with other microservices for new tasks; adding to that, microservices workflow can be shared among tasks. Fig. 1 shows a microservice provisioning use case. Here, workflows A and B each contain three microservices (A1–A3 and B1–B3). Each microservice type, indicated by its color, has several candidate operators with concrete implementations. For example, the blue microservices have three operator candidates. Operators to execute a microservice can be instantiated on any device in the workflow's path—this enables resource sharing. For example, operator 4 implements A1 of workflow A on device 1. The same operator also implements B1 of workflow B.

In mission-critical applications where accurate awareness and rapid response are essential (e.g., tracking humans trapped in fires), the workflow of microservices must ensure satisfaction with timing and accuracy constraints. To satisfy tasks with different delays and accuracy requirements, judicious selection, provisioning, composition, and instantiation of microservices on the distributed infrastructure of nodes is necessary. To address latency/resource constraints, we exploit the fact that a microservice may be realized by distinct operators with varying cost and performance characteristics. Consider an image-based fire detection microservice. An operator implementing DNN object detection models (e.g., YOLO [14]) can provide accurate results but will consume more time and resources compared to another operator with simple image processing techniques based on color histograms. We leverage such trade-offs among operators of the same microservices to meet the time, resource, and accuracy needs of application workflows. Sharing operators (and thus microservices) among different workflows can further help optimize analytics deployment to achieve the maximal QoS and good resource utilization.

Fig. 2 illustrates the structure of AMPHI, in which our proposed microservice provisioning framework handles various workflows requested by users through optimal operator instantiation and sharing. Within the system, workflows are generated during each time epoch from a user interface aiming to conduct data analytics tasks for different purposes. The *microservice provisioning manager* makes appropriate decisions on operator selection, placement, and instantiation–the choice is based on current device and network conditions obtained from a *system status tracker*. Once operators are appropriately deployed, the configuration of operators required for the workflow is instantiated. IoT data generated by sensors are then transmitted to and processed by corresponding operators following the configuration to get the data analysis results.

B. Related Work in Microservices and Operator Placement

Microservice composition and deployment have been studied in the context of heterogeneous cloud/fog/edge computing environments in literature, whose main focus is reducing the system loads [15]–[17] and service delays [18]–[20] using efficient scheduling and deployment strategies. Techniques to generate custom IoT deployment in the cloud [21], to minimize average waiting time in the edge-cloud environment using reinforcement learning [22], and to allocate resources among edge devices via dynamic scheduling [23] have been explored. In most cases above, microservices selections are identical; a key difference in AMPHI is that it chooses among diverse operators, i.e., realizations for a microservice, that can cater and adapt to infrastructure variations and application QoS needs. Another related problem is operator placement, where operators are chosen to meet QoS objectives [24]–[26] and resource constraints [27], [28]. Techniques include splitting IoT applications into a sequence of operators in a fog-computing platform using greedy approaches [29] and resource-aware

frameworks to dynamically select sensing strategies to execute at the edge using reinforcement learning [30]. Furthermore, scheduling for time-constrained processing of data streams among fog-cloud nodes [31] and heuristic methods for operator placement in stream processing settings [32] have also been studied. Some existing works [22], [27] have employed reinforcement learning approaches to develop deployment strategies. However, adapting RL-based models to dynamic and time-varying application scenarios proves challenging. Additionally, these models struggle to accommodate the diversity of devices and application requirements in our scenarios. Besides, above works focus on sharing a predetermined set of operators across applications; in contrast, AMPHI leverages the possibility of using operators with diverse cost and performance characteristics to implement microservices along with optimizing the shared use of the selected microservices.

III. Microservice Provisioning Problem

In this section, we formulate our Microservice Provisioning (MSP) problem, which aims to provide optimal operator selection and instantiation across heterogeneous devices at the beginning of each epoch, spanning from t_0 to t_τ.

A. Notations

We define the device set as $\mathcal{D} = \{d_1, d_2, ...\}$. The computing resources of each device, denoted by $Res^n(d_i)$ with $n \in [1, N]$, represent the number of different types of computing resources, such as CPU and memory. We monitor the remaining power of each device, denoted as $Pwr(d_i)$. At the beginning of each epoch, we predict the trajectory of each device as $Traj(d_k) = [l_0^k, l_1^k, ..., l_\tau^k]$, indicating the spatial location of device d_k at each timestamp within $[t_0, t_\tau]$. Stationary devices have constant coordinates throughout the epoch. We estimate each device's reliable communication range, $CR(d_k)$, (to access points) based on historical measurements. Given device trajectory $Traj(d_k)$, we can determine the time slots when it has a network connection by checking if its starting and ending locations fall within the device's communication range. This allows us to obtain the set of effective time slots, denoted as $ET(d_k) = \{l_n^k | l_n^k \in Traj(d_k); MinD(l_n^k), MinD(l_{n+1}^k) < CR(d_k)\}$, where $MinD(l_n^k)$ computes the distance between location l_n^k and its nearest access point. Furthermore, we can compute the time ratio when two devices, d_k and $d_{k'}$, have a connection through access points over each epoch by $ETR(d_k, d_{k'}) = |ET(d_k) \cap ET(d_{k'})|/\tau$.

We consider a set of user-provided workflows in each epoch, denoted as $\mathcal{W}$. Each workflow $w_h \in \mathcal{W}$ is associated with a source device D_h^0 and a data sink device D_h^* in $\mathcal{D}$, representing the sensor data source and the final result destination. We use λ_h to represent the data emission rate of the sensor linked to each workflow and assign a data processing delay tolerance $DT(w_h)$ to limit its processing time based on the emergency levels of monitoring requirements. Each workflow is characterized by a microservice sequence $\mathcal{M}(w_h) = [m_h^1, m_h^2, ...]$, depicting its sequential data processing requirements. Here, m_h^k represents the kth microservice in the sequence. All microservices within each workflow have the same data arrival rate, denoted by $\lambda(m_h^k) = \lambda_h$. We use microservice type $MType(m_h^k)$ to indicate m_h^k's data processing objective, e.g., fire detection. The set of microservices required by all workflows is denoted as $\mathcal{M} = \bigcup_{w_h \in \mathcal{W}} \mathcal{M}(w_h)$. For brevity, we write each microservice as m_i, with the option to revert to m_h^k when needed.

Multiple implementation options are available for each type of microservice $MType(m_i)$, reflecting diverse approaches facilitated by operators. We use set $\mathcal{O}(m_i)$ to denote operator candidates to implement microservice m_i, serving its corresponding microservice type $OType(o_j) = MType(m_i)$, where $o_j \in \mathcal{O}(m_i)$. Additionally, we use $\mathcal{O} = \bigcup_{m_i \in \mathcal{M}} \mathcal{O}(m_i)$ to represent all operator candidates for implementing all microservices. Different operator candidates for the same microservice may yield varying data processing results. We evaluate the data processing quality of each operator candidate $o_j \in \mathcal{O}$ using accuracy $A(o_j) \in [0, 1]$ based on measurement studies. To implement a workflow, we must select and instantiate an operator for each of its microservices at a specific device. Illustrating this decision-making process, we introduce the boolean decision matrix $\mathbf{X} = [x_{ijk}]_{|\mathcal{M}| \times |\mathcal{O}| \times |\mathcal{D}|}$, where $x_{ijk} = 1$ iff operator $o_j \in \mathcal{O}$ is chosen to implement microservice $m_i \in \mathcal{M}$ and instantiated on device $d_k \in \mathcal{D}$.

B. Workflow Fulfillment Evaluations

Given a microservice provisioning decision $\mathbf{X}$, we aim to evaluate its impact on workflow fulfillment by assessing both the accuracy of data processing and the total delay for each workflow. The workflow's accuracy, denoted as $Acc(w_h, \mathbf{X})$, is computed by:

$$Acc(w_h, \mathbf{X}) = \prod_{m_i \in \mathbf{M}(w_h)} \Big(\sum_{o_j \in \mathcal{O}} \sum_{d_k \in \mathcal{D}} x_{ijk} \times A(o_j) \Big). \tag{1}$$

We also evaluate the delay of each workflow, denoted as $Delay(w_i, \mathbf{X})$, as follows:

$$Delay(w_h, \mathbf{X}) = T_{tran}(w_h, \mathbf{X}) + T_{proc}(w_h, \mathbf{X}), \tag{2}$$

where $T_{tran}(w_h, \mathbf{X})$ represents the total delay of workflow w_h from its source device to the data sink, computed as:

$$\begin{aligned} T_{tran}(w_h, \mathbf{X}) = {} & TD(D_h^0, D(m_h^1)) + TD(D(w_h^{|\mathcal{M}(w_h)|}), D_h^*) \\ & + \textstyle\sum_{k=1}^{|\mathbf{M}(w_h)|-1} TD(D(m_h^k), D(m_h^{k+1})), \end{aligned} \tag{3}$$

where $D(m_i) = \sum_{d_k \in \mathcal{D}} k \times (\sum_{o_j \in \mathcal{O}} x_{ijk})$, with $m_i \in \mathcal{M}$ indicates the device where the microservice m_i is implemented. $TD(D(m_h^k), D(m_h^{k+1}))$ represents the data transmission delay from the device where the operator of m_h^k is mapped, to the device where its consecutive operator is deployed.

We compute $T_{proc}(w_h, \mathbf{X})$, the processing time of workflow w_h, by summing the processing times of each microservice:

$$T_{proc}(w_h, \mathbf{X}) = \sum_{m_i \in \mathbf{M}(w_h)} \sum_{o_j \in \mathcal{O}} \sum_{d_k \in \mathcal{D}} x_{ijk} PT(o_j, d_k, \lambda'(o_j)), \tag{4}$$

where $PT(o_j, d_k, \lambda'(o_j))$ denotes the expected data processing delay of operator o_j on device d_k using the M/M/1 queuing model. This is computed using the measured average data

processing rate $PR(o_j, d_k)$ for operator o_j on device d_k and the data arrival rate at operator o_j, computed using $\mathbf{X}$ by: $\lambda'(o_j) = \sum_{m_i \in \mathcal{M}} \sum_{d_k \in \mathcal{D}} x_{ijk} \lambda(m_i)$.

Last, a utility function is defined to evaluate the fulfillment of each workflow as follows:

$$\mathbf{\Theta}(w_h, \mathbf{X}) = \alpha_1 \frac{Acc(w_h, \mathbf{X}) - Acc_{w_h}^{min}}{Acc_{w_h}^{max} - Acc_{w_h}^{min}} + \alpha_2 (1 - \frac{Delay(w_h, \mathbf{X})}{DT(w_h)}) \tag{5}$$

This utility function integrates the accuracy of the workflow, normalized between its minimum and maximum accuracy represented by $Acc_{w_h}^{min}$ and $Acc_{w_h}^{max}$, across all candidate operators. It also includes a penalty function for the workflow's delay, which decreases as the delay surpasses the workflow's tolerance $DT(w_h)$. The parameters α_1 and α_2 represent the weights assigned to these metrics, summing up to 1.

C. Microservice Provisioning Optimization

We formulate our MSP problem to optimize the selection and deployment decisions $\mathbf{X}$ of operators to maximize the overall utility function of all workflows over each epoch. The formulation is written as:

$$\max \sum_{w_h \in \mathcal{W}} \mathbf{\Theta}(w_h, \mathbf{X}) \quad \text{s.t.} \tag{6a}$$

$$\sum_{o_j \in \mathcal{O}} \sum_{d_k \in \mathcal{D}} x_{ijk} \leq 1, \quad \forall m_i \in \mathcal{M} \tag{6b}$$

$$\sum_{d_k \in \mathcal{D}} x_{ijk} \, OType(o_j) = MType(m_i), \quad \forall m_i \in \mathcal{M}, \forall o_j \in \mathcal{O} \tag{6c}$$

$$\sum_{m_i \in \mathcal{M}} \lambda(m_i) x_{ijk} \leq PR(o_j, d_k), \quad \forall o_j \in \mathbf{O}, \forall d_k \in \mathcal{D} \tag{6d}$$

$$\sum_{m_i \in \mathbf{M}} \sum_{o_j \in \mathbf{O}} RqRes^n(o_j, d_k, \lambda'(o_j)) x_{ijk} \leq Res^n(d_k), \quad \forall d_k \in \mathcal{D}, \forall n \in [1, N] \tag{6e}$$

$$\sum_{m_i \in \mathcal{M}} \sum_{o_j \in \mathcal{O}} x_{ijk} RqPwr(o_j, d_k, \lambda'(o_j)) \leq Pwr(d_k), \forall d_k \in \mathcal{D} \tag{6f}$$

$$ETR(D(m_h^k), D(m_h^{k+1})) \geq \beta, \forall w_h \in \mathcal{W}, k < |\mathcal{M}(w_h)| \tag{6g}$$

$$ETR(D_h^0, D(m_h^1)), \; ETR(D(m_h^{|\mathcal{M}(w_h)|}, D_h^*) \geq \beta, \; \forall w_h \in \mathcal{W}. \tag{6h}$$

Several constraints are employed in Eq. (6) to ensure the proper operation of microservices. Constraint (6b) guarantees that each microservice is implemented by only one operator on a single device. Constraint (6c) ensures that the chosen operator for each microservice matches its type. Additionally, Constraint (6d) governs data flow, preventing bottlenecks by ensuring that the data arrival rate at each operator does not exceed its processing rate. Constraints (6e) and (6f) require that each selected operator must be instantiated on a device with adequate computing resources and all devices have sufficient remaining power to handle their workload during an epoch, respectively. Here, $RqRes^n(o_j, d_k, \lambda'(o_j))$ and $RqPwr(o_j, d_k, \lambda'(o_j))$ represent the computing resource requirement and power consumption rate for instantiating operator o_j on device d_k. $\lambda'(o_j)$ indicates the data arrival rate at o_j, as defined in Eq. (4). Constraints (6g) and (6h) enforce that, for each workflow, the network connection from the source device to the sink device remains consistently connected throughout an epoch, surpassing a time ratio threshold β. Note that, $D(m_h^k)$ is defined in Eq. (3).

The NP-hardness of MSP can be proven by reducing the classical Knapsack problem to its special case, where only one device is available, and each workflow consists of a single microservice mapped to one operator candidate. The objective is to maximize the sum of utility functions of all microservices instantiated on the devices with limited computing resources.

IV. AMPHI Approaches

This section outlines the key components supporting AMPHI's microservices management and the decision-making approaches it employs for microservice provisioning.

System status tracking and operator preparation. Before reaching a satisfactory decision, AMPHI gets the available devices' status (e.g., available computing resources, battery level, location, network condition, etc.). This can be achieved by using pub-sub mechanisms such as MQTT to make each device publish its status periodically to the system status tracker. After that, AMPHI extracts all types of microservices required by the workflows and pulls their corresponding operator images (from which an operator can be instantiated) to each device from the operator registry, ensuring rapid instantiation.

Microservice provisioning algorithms. Based on system status, AMPHI next determines the appropriate mapping of operators to microservices across all workflows so that QoS objectives can be maximized. Given the NP-hardness of the MSP problem and the real-time decision-making demands in our mission-critical IoT environments, we design multiple heuristic algorithms with varying time-objective priorities to expedite microservice provisioning.

1) Greedy-based algorithms: For each microservice, two options exist for assigning an operator: sharing with a compatible instantiated one or instantiating a new one. We use the term *operator-device mapping* to describe an operator instantiated/to be instantiated on a specific device. We first propose algorithms employing a greedy approach, where microservices greedily select among operator-device mappings based on three strategies: (i) *Accuracy-First* prioritizes accuracy by selecting the operator-device mapping with the highest accuracy among all feasible options; (ii) *Delay-First* prioritizes delay reduction by choosing the mapping with the lowest delay increment computed as the sum of network delay between the operator and its previous operators' host devices, along with the operator's processing time; (iii) *Greedy-MultiCriteria* strives for a balance between accuracy and delay by employing TOPSIS [33] to calculate the similarity of each solution to the ideal solution on both accuracy and delay increment, and select the closest mapping in terms of both criteria.

2) Meta-heuristic algorithms: Solutions generated by greedy-based heuristics often exhibit limited performance. Thus, we propose two meta-heuristic approaches to improve this, considering their 'anytime' property for meeting timely computing requirements. These approaches, named *AMPHI-LS* and *AMPHI-ILS*, are inspired by the concepts of local search and iterated local search, respectively. Since both share common intermediate processes, we provide their details in

Algorithm 1, with *AMPHI-ILS* incorporating additional steps from lines 18 to 21.

In *AMPHI-LS*, we initially generate a solution (lines 3 to 10) by employing our greedy-based algorithm, specifically *Greedy-MultiCriteria*, chosen for its tendency to achieve a relatively balanced objective. Subsequently, we enhance this initial solution by exploring neighboring solutions using various exploration strategies (lines 12 to 16). These strategies, tailored to our microservice provisioning problem, include: *swapDevice*, which exchanges operators instantiated on two different devices; *moveOperator*, relocating a single operator to a new device; and *changeOperator*, replacing a single operator with another operator implementing the same type of microservice. Throughout the exploration, we filter out any generated neighbors violating constraints specified in Eq. (6) to ensure the selection and instantiation of valid operators. The termination of neighbor exploration occurs when either the number of successive non-improvements exceeds a predetermined limit or the time budget is depleted, at which point the best-known solution is returned (lines 11 and 24).

While *AMPHI-LS* efficiently enhances solutions, its incremental approach makes it susceptible to producing local optima. To address this issue, we propose *AMPHI-ILS*, which incorporates perturbations during neighbor exploration to boost the probability of escaping local optima (lines 18 to 21) *AMPHI-ILS* shares the steps with *AMPHI-LS* but introduces perturbations into the generated solution (best neighbor) if neighbor exploration fails to yield improvement at each improvement iteration, additionally restarting the search process. Specifically, the perturbation is conducted by randomly relinking a data flow from one operator to another by replacing the endpoint operator with another operator of the same type.

Algorithm 1: AMPHI Metaheuristic

Input: Workflows $\mathcal{W}$, Devices $\mathcal{D}$, Operators $\mathcal{O}$, Type $\mathcal{H}$

1 $n \leftarrow maxNoImprovement$; $t \leftarrow maxRunningTime$;
2 $counter \leftarrow 0$; $\mathbf{X} \leftarrow \{\}$;
// Greedily generate initial solution
3 **for** $w_h \in \mathcal{W}$ *and* $m_h^k \in \mathcal{M}(w_h)$ **do**
4 $\quad \Omega \leftarrow instantiated(m_h^k)$ // Operator-device instantiated
5 $\quad$ **for** $o_j \in \mathbf{O(m_n^k)}$ *and* $d_k \in \mathbf{D}$ **do**
6 $\quad\quad \Omega \leftarrow \Omega \cup < o_j, d_k >$ // Get operator-device mapping
7 $\quad \Omega \leftarrow$ filterByConstraints(Ω);
8 $\quad \Omega \leftarrow$ sortByGreedyObjective(Ω, AMPHI-GDM);
9 $\quad \mathbf{X}[m_h^k] \leftarrow \Omega[0]$; // Get the best mapping
10 $\quad$ updateResource(Ω) ; // Update resource usage
11 **while** $counter < n$ *or* $runningTime < t$ **do**
12 $\quad V \leftarrow \sum_{w_h \in \mathcal{W}} \mathbf{\Theta}(w_h, \mathbf{X})$ based on Eq.(5) ;
$\quad$ // Improve X through neighboring exploration
13 $\quad \mathbf{X} \leftarrow$ *Best neighbor of swapDevice(X)*;
14 $\quad \mathbf{X} \leftarrow$ *Best neighbor of moveOperator(X)*;
15 $\quad \mathbf{X} \leftarrow$ *Best neighbor of changeOperator(X)*;
16 $\quad V' \leftarrow \mathbf{X}$;
17 $\quad$ **if** $V' \leq V$ **then** // If no improvement, apply perturbation
18 $\quad\quad E \leftarrow$ All links among microservices;
19 $\quad\quad < m_1, m_2 > \leftarrow$ getRandomDataLinks(E);
20 $\quad\quad OD \leftarrow$ All $< o_j, d_k >$ pairs, with o_j is not chosen for m_2 and $OType(o_j)$=$MType(m_2)$
21 $\quad\quad \mathbf{X}[m_2] =$ getRandomItemFrom(OD);
22 $\quad\quad counter \leftarrow counter + 1$
23 $\quad$ **else** $counter \leftarrow 0$;
24 **return** $\mathbf{X}$;

V. Evaluations

In this section, we assess AMPHI using both real-world testbeds and simulations. We first conduct a thorough measurement study to build regression models for estimating operators' resource and power consumption and gather device and network parameters as input for our MSP problem. Subsequently, we evaluate our proposed algorithms in a real testbed. Additionally, to investigate the scalability of our algorithms, we simulate high-rise fire scenarios to generate workflows and device mobility, examining algorithm performance across varying numbers of devices and workflows.

A. Testbed and Measurement Study

Fig. 3 displays our testbed, consisting of three diverse edge servers, two IoT devices, and a simulated high-rise fire scenario (a poster with human, fire, and windows). The edge servers are (i) a Raspberry Pi 4B, (ii) an NVIDIA Jetson Nano, and (iii) an NVIDIA Jetson AGX Xavier. The IoT devices are two Raspberry Pi 4Bs with Arducam 5MP cameras. One is stationary with a plugged-in power supply, while the other is mobile with a PiJuice portable power supply mounted on a drone. The Raspberry Pis are running on 64-bit Raspbian OS, while the Jetson Nano is running on Ubuntu 18.04, and the

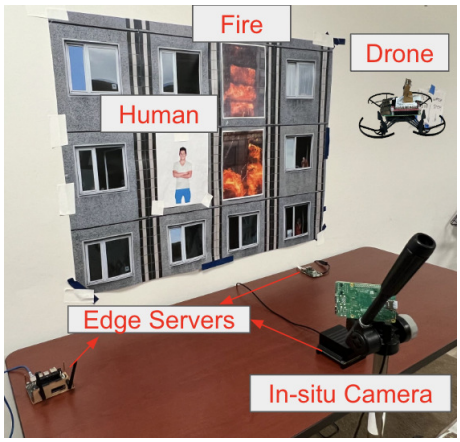


Fig. 3. Testbed including edge servers and IoT devices.

Jetson Xavier is running on Ubuntu 20.04. We use Linux's traffic control to simulate network delays ranging from 10 to 100 ms. We allocate 20% of computing resources for each device's system operation during operator instantiation. Management and monitoring are handled by Docker Swarm [34] and cAdvisor [35]. We developed 13 operators using Docker to handle 5 types of microservices for image processing in high-rise fire monitoring scenarios: (i) Fire Detection, (ii) Human Detection, (iii) Pose Estimation, (iv) Background Subtraction, and (v) Color Filtering. Operators exchange data via HTTP requests, each wrapped in a Flask framework. We

TABLE I
MEASUREMENT RESULTS OF SELECTED OPERATORS

Microservices Type		Human Detection		Fire Detection	
Operators		TinyYOLO	YOLO	TinyYOLO	YOLO
Accuracy (%)		34.3	56.8	32.1	51.9
Processing Rate(pkg./s)	Raspi	0.96	0.13	1.06	0.14
	Nano	1.63	0.22	1.81	0.23
	Xavier	2.12	0.37	2.38	0.41
CPU Usage (%)		32.7	106.4	29.9	86.8
Memory Usage (MB)		654	982	527	845
Power (W)		0.80	1.54	0.78	1.52

perform a measurement study by running our operators on devices in our testbed under varying data arrival rates to model different device and operator characteristics. We show part of our measurement results by displaying the data of two human detection operators and two fire detection operators, both differentiated by the model complexity (TinyYOLO and YOLO [14]).

1) Performance Measurement: We quantify operators' performance using accuracy and data processing rate (processing time per data unit). The accuracy is calculated by the mean Average Precision (mAP), a standard metric for evaluating the accuracy of object detection algorithms. The mAP for human detection and fire detection operators are evaluated on the COCO [36] and FireNet [37] datasets respectively. Regarding processing rate, we record the time taken to process a single 1 MB image data (in seconds) for each operator on three different devices and calculate the reciprocal. The measurement results are presented in Table I.

2) Resource Consumption Measurement and Modeling: We measured the resource consumption of each operator-device mapping under varying data arrival rates. The metrics we considered include: i) CPU usage percentage (measured using the 'docker stats' command), ii) physical memory usage (measured using the 'docker stats' command), and iii) power consumption rate (in W), determined by the difference in power usage between idle and operator execution states, as measured using a USB power meter. We measured these three metrics across data arrival rates from 0.05 to 1 (in packages per second). We observed a consistent non-linear increase in CPU usage and power consumption with rising data arrival rates, while memory usage remained constant. Based on these observations, we employ the following regression models to characterize resource consumption:

$$RqRes^n(o_j, d_k, \lambda'(o_j)) = \begin{cases} a_{j,k}\lambda^{b_{j,k}} + c_{j,k}, & \text{if } n \text{ indicates CPU;} \\ c'_{j,k}, & \text{if } n \text{ indicates memory,} \end{cases}$$

where $\lambda'(o_j)$ represents the data arrival rate at o_j, n signifies the resource type, and $a_{j,k}$, $b_{j,k}$, $c_{j,k}$, and $c'_{j,k}$ are parameters derived from our regression analysis. The power consumption model $RqPwr^n(o_j, d_k, \lambda'(o_j))$ follows a similar structure to the CPU model. The measurement results and fitted curves for CPU usage and power consumption are depicted in Fig. 4 (memory usage data is not included due to space limit). We computed the R-squared values for all models, resulting in values of 0.9888 for CPU, 0.9874 for memory, and 0.9867 for power, indicating a strong fit of the derived models to the data. Table I displays the CPU usage, memory usage, and power consumption of selected operators at a data arrival rate of 1 package per second.

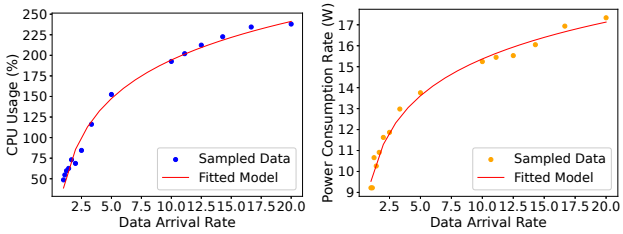


Fig. 4. Modeling of CPU usage and power consumption for TinyYOLO human detection operator on Jetson Xavier.

B. Experiments and Results

Testbed setup. We implement AMPHI on our testbed to evaluate different algorithms. Alongside the greedy algorithms proposed in Sec. IV, we implement two state-of-the-art approaches [32]: *ODP-LS* (Local Search) and *ODP-TS* (Tabu Search) for solving the operator deployment problem as baselines. We conduct experiments with different numbers of workflows, from 1 to 15, selected uniformly from the following microservice sequence templates: i) *Background Subtraction* followed by *Fire Detection*, ii) *Background Subtraction* followed by *Human Detection* and then *Pose Estimation*, iii) *Color Filtering* followed by *Fire Detection*, and iv) *Fire Detection* followed by *Human Detection*. In addition, We uniformly generate workflows with data emission rates ranging from 0.02 to 0.2, each with an average package size of 1.2 MB. To prevent network congestion, we allocate 30% of the link bandwidth. For fair comparison among the four meta-heuristic algorithms, we set the maximum non-improvement times to 20 and the maximum running time to 1 hour for each epoch. The metrics used to evaluate algorithm performance include (i) average objective function value, (ii) average accuracy, (iii) average delay, (iv) CPU usage, (v) memory usage, and (vi) number of satisfied workflows (when all microservices are mapped with an operator).

Testbed results. Figs. 5 and 6 illustrate the algorithm comparison results of real-world testbed experiments. Figs.5a to 5d report the QoS achieved by different algorithms, while Figs. 6a to 6b detail the resource utilization. From the results, it's evident that greedy-based approaches outperform others in their respective greedy objectives. However, they struggle to achieve a good trade-off among different metrics. For example, while the *Accuracy-First* outperforms others in average accuracy (Fig. 5b), it exhibits poor performance in average delay (Fig. 5c). Although *Greedy-MultiCriteria* achieves a better accuracy-delay trade-off compared to other greedy approaches, it still falls short compared to our proposed meta-heuristic algorithms (*AMPHI-LS* and *AMPHI-ILS*), as shown in Fig. 5a. From Fig. 5a, it's evident that all four meta-heuristic algorithms exhibit similar performance in scenarios with relatively few workflows, attributed to sufficient

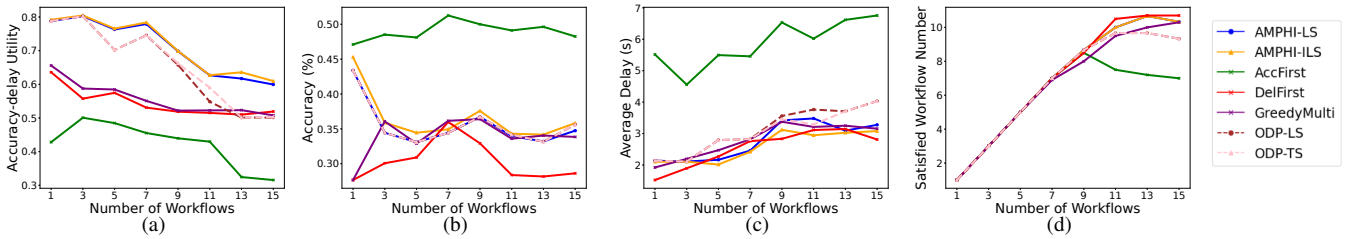


Fig. 5. Testbed results for (a) accuracy-delay utility, (b) workflow accuracy, (c) workflow delay, and (d) satisfied number of workflows.

computing resources. However, as the number of workflows increases, the performance gap widens. For example, when there are 15 workflows, AMPHI approaches outperform ODP by up to 26.8% in average accuracy-delay utility. Figs. 5b and 5c demonstrate the enhanced accuracy and reduced delay achieved by our AMPHI meta-heuristic algorithms compared to ODP algorithms. Fig.5d illustrates that as the number of workflows increases, which overwhelms the devices' computing capacity, AMPHI meta-heuristic algorithms can satisfy more workflows than ODP algorithms while achieving higher objectives simultaneously. Additionally, Figs. 6a and 6b demonstrate that AMPHI meta-heuristic algorithms exhibit better resource efficiency compared to ODP algorithms, with reductions of up to 17.4% in CPU usage and 66.7% in memory usage. Particularly in Fig. 6a, *AMPHI-LS* consumes fewer CPU resources but achieves higher accuracy-delay utility than ODP approaches.

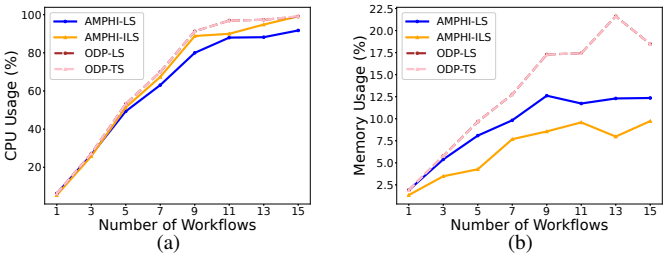


Fig. 6. Testbed results for (a) CPU usage and (b) memory usage across all the devices.

Simulation setup. We expand the evaluations of AMPHI using a simulator with a broader parameter range. Leveraging a simulated high-rise fire setting described in the literature [7], specifically in the DBH building at UCI campus, we simulate drone mobility using waypoint scheduling approaches [7] to fulfill various monitoring tasks generated based on firefighting guidelines and simulated fire conditions. Additionally, we introduce stationary sensing units and edge servers with diverse computing capabilities. The device and operator profiles are derived from real-world measurements. We scale the number of devices to 50, maintaining a ratio of stationary IoT devices, mobile IoT devices, and edge servers at 6:3:1. For edge servers, we uniformly select device models from Jetson Xavier, Jetson Nano, and Raspberry Pi, while IoT devices are chosen from Nano and Pi. We vary each sensor's data emission rate from 0.02 to 0.2 and the data package size from 100 KB to 2 MB. Network delay among devices ranges from 100 to 500 ms, and bandwidth ranges from 10 to 20 Mbps. We establish the network infrastructure by distributing WiFi access points within a 15-meter distance in the simulation scene, covering all stationary devices, while mobile devices' communication ranges vary with a mean of 10 meters to account for building interference. We increase the number of workflows from 10 to 80. Mobile devices are mapped to drones, and workflows utilizing these devices as data sources are mapped to monitoring tasks for fire and human detection. Simulations run on an Ubuntu 22.04 Intel Workstation with a 13700K CPU and 32 GB DDR5 memory.

Simulation results. Fig. 7 presents our experimental findings. We focus on the performance of our proposed algorithms *AMPHI-LS* and *AMPHI-ILS*, as well as *ODP-LS* and *ODP-TS*, which yield the highest accuracy-delay utility and omit the results of greedy algorithms for clarity. The metrics comparison from Fig. 7a to Fig. 7f aligns with our testbed results, demonstrating that AMPHI meta-heuristics exhibit excellent scalability and outperform baseline algorithms as device and workflow numbers scale up. We evaluate the running time of our AMPHI meta-heuristic approaches as the number of devices increases from 10 to 50 and the number of workflows increases from 10 to 50. For *AMPHI-ILS*, the running time ranges from 30.73s to 200.38s as the workflow number grows from 10 to 50 when there are 50 devices. When the workflow number is fixed at 10, the running time increases from 1.56s to 40.75s as the number of devices changes from 10 to 50. For *AMPHI-LS*, the running time are 3.03-15.01s and 0.08-4.09s with the same setting. It's noteworthy that *AMPHI-ILS*, which aims to approach global optima requires more time than *AMPHI-LS*, which provides sub-optimal solution relatively. We regard this running time gap as a reasonable cost since the majority of microservices are not continuously changing over the time. In drone-assisted high-rise fire monitoring, for example, the drones need to observe specific locations for a certain duration to ensure sufficient information is gathered. During this time window, services are not expected to be reconfigured within a very short time. Even if the configuration need to be changed within constrained time-frames for emergent tasks, *AMPHI-LS* would be qualified for making timely decisions.

VI. Conclusion

In this paper, we present AMPHI, a microservice provisioning framework to efficiently handle data analytics tasks in hybrid IoT environments with stationary and mobile sensing platforms, including autonomous ground and aerial vehicles. AMPHI aims to optimize operators' selection, deployment,

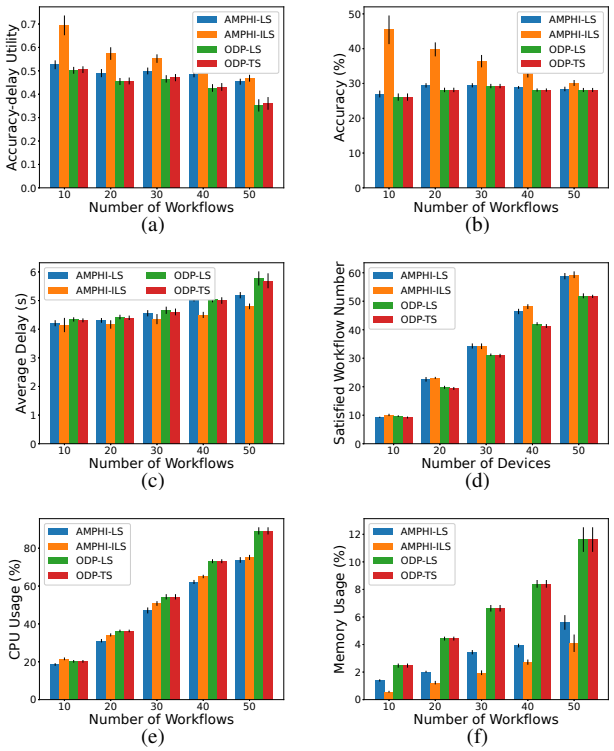


Fig. 7. Simulation results: (a) accuracy-delay utility, (b) accuracy, (c) delay, (d) satisfied workflows, (e) CPU usage and (f) memory usage.

and instantiation for the required microservice workflows for time-sensitive applications considering operator, device, and mobility characteristics. We illustrate the effectiveness of our approach through detailed experiments and measurement studies of the AMPHI implementation in a real testbed and a simulator using high-rise fire scenarios. Our ongoing work explores techniques to improve the robustness of AMPHI to deal with unexpected events such as device failures, network disruptions, and power outages. Future directions include capabilities in a device-to-cloud continuum and the role of reusable microservices in long-term analytical applications. Approaches with more advanced and intelligent backbones like reinforcement learning are supposed be explored to provide fine-grained and timely control; we expect that frameworks such as AMPHI that allow reconfigurability of the underlying hardware-software platforms as technologies evolve are critical to the longevity of IoT deployments.

Acknowledgement

This research was supported by the UC Laboratory Fees Program (UCOP Grant ID LFR-20-653572), NSF Grants No. 2133391, No. 2008993), and DARPA (No. FA875016-2-0021). Any opinions, findings, and conclusions expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

[1] K. Benson *et al.*, "Scale: Safe community awareness and alerting leveraging the internet of things," *IEEE Commun. Mag.*, vol. 53, no. 12, 2015.
[2] R. De Michele and M. Furini, "Iot healthcare: Benefits, issues and challenges," in *GoodTechs*, 2019.
[3] N. Gondchawar *et al.*, "Iot based smart agriculture," *IJARCCE*, vol. 5, no. 6, 2016.
[4] Y. Liu *et al.*, "Two-layer routing for high-voltage powerline inspection by cooperated ground vehicle and drone," *Energies*, vol. 12, no. 7, 2019.
[5] N. Iversen *et al.*, "Design, integration and implementation of an intelligent and self-recharging drone system for autonomous power line inspection," in *IROS*, 2021.
[6] O. B. Schofield *et al.*, "Cloud to cable: A drone framework for autonomous power line inspection," in *DSD*, 2020.
[7] F. Liu *et al.*, "Dragonfly: Drone-assisted high-rise monitoring for fire safety," in *SRDS*, 2021.
[8] C. Pu and P. Zhu, "Defending against flooding attacks in the internet of drones environment," in *GLOBECOM*, 2021.
[9] R. Tariq *et al.*, "Dronaid: A smart human detection drone for rescue," in *HONET-ICT*, 2018.
[10] T. Salah *et al.*, "The evolution of distributed systems towards microservices architecture," in *ICITST*, 2016.
[11] M. Waseem *et al.*, "A systematic mapping study on microservices architecture in devops," *Journal of Systems and Software*, vol. 170, 2020.
[12] E. Al-Masri, "Enhancing the microservices architecture for the internet of things," in *Big Data*, 2018.
[13] P. Bellavista and A. Zanni, "Feasibility of fog computing deployment based on docker containerization over raspberrypi," in *Proc. 18th Int. Conf. Distrib. Comput. & Netw.*, 2017.
[14] "YOLOv5 by Ultralytics," https://github.com/ultralytics/yolov5, 2020.
[15] Z. Ding *et al.*, "Kubernetes-oriented microservice placement with dynamic resource allocation," *IEEE Trans. Cloud Comput.*, 2022.
[16] F. Wan *et al.*, "Chain-oriented load balancing in microservice system," in *WCCCT*, 2020.
[17] J. Cui *et al.*, "A learning-based dynamic load balancing approach for microservice systems in multi-cloud environment," in *ICPADS*, 2020.
[18] J. Huang *et al.*, "Qos correlation-aware service composition for unified network-cloud service provisioning," in *GLOBECOM*, 2016.
[19] Z. Liu *et al.*, "Proactive recommendation of composite services in multi-access edge computing," *IEEE Trans. Serv. Comput.*, 2023.
[20] Y. Zhang *et al.*, "Deadline-aware dynamic task scheduling in edge–cloud collaborative computing," *Electronics*, vol. 11, no. 15, 2022.
[21] M. Vögler *et al.*, "Diane-dynamic iot application deployment," in *International Conference on Mobile Services*, 2015.
[22] L. Chen *et al.*, "Iot microservice deployment in edge-cloud hybrid environment using reinforcement learning," *IEEE IoTs Journal*, vol. 8, no. 16, 2020.
[23] A. Samanta *et al.*, "Dyme: Dynamic microservice scheduling in edge computing enabled iot," *IEEE IoTs Journal*, vol. 7, no. 7, 2020.
[24] S. Forti *et al.*, "Lightweight self-organising distributed monitoring of fog infrastructures," *Future Generation Computer Systems*, vol. 114, 2021.
[25] W. Ma *et al.*, "Multi-objective microservice deployment optimization via a knowledge-driven evolutionary algorithm," *Complex & Intelligent Systems*, vol. 7, 2021.
[26] C.-Y. Hsieh *et al.*, "T2c: A multi-user system for deploying dnns in a thing-to-cloud continuum," in *MSN*, 2022.
[27] Z. Nezami *et al.*, "Decentralized edge-to-cloud load balancing: Service placement for the internet of things," *IEEE Access*, vol. 9, pp. 64 983–65 000, 2021.
[28] M. Siew *et al.*, "Let's share vms: Optimal placement and pricing across base stations in mec systems," in *INFOCOM*, 2021.
[29] H.-J. Hong *et al.*, "Supporting internet-of-things analytics in a fog computing platform," in *CloudCom*, 2017.
[30] P. Venkateswaran *et al.*, "Ream: Resource efficient adaptive monitoring of community spaces at the edge using reinforcement learning," in *SMARTCOMP*, 2020.
[31] P. Ntumba *et al.*, "Scheduling continuous operators for iot edge analytics," in *EdgeNet*, 2021.
[32] M. Nardelli *et al.*, "Efficient operator placement for distributed data stream processing applications," *IEEE TPDS*, vol. 30, no. 8, 2019.
[33] R. Rahim, *et al.*, "Technique for order of preference by similarity to ideal solution (topsis) method for decision support system in top management," *Pro Mark*, vol. 8, no. 2, 2018.
[34] "Docker Swarm," https://docs.docker.com/reference/cli/docker/swarm/.
[35] "cAdvisor," https://github.com/google/cadvisor.
[36] "COCO," https://github.com/cocodataset/cocodataset.github.io.git, 2014.
[37] "FireNet," https://github.com/OlafenwaMoses/FireNET, 2019.